

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**



ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И УПРАВЛЯЮЩИХ СИСТЕМ

Лабораторная работа №3
по дисциплине: **Операционные системы**
тема: **«Файловые системы в ОС Linux: сравнение, области эффективности.
Виртуальная файловая система. Пользовательская файловая система»**

Выполнил: ст. группы ВТ-231
Масленников Д. А.

Проверили:
Островский А. М.

Белгород 2025 г

Лабораторная работа № 3

Цель работы: Изучить популярные файловые системы в ОС Linux (ext4, Btrfs, ReiserFS, NTFS,

FAT32), определить область эффективности каждой из них, разобраться как осуществляется работа с виртуальной файловой системой (VFS) ОС Linux и выполнить разработку пользовательской файловой системы в соответствии с индивидуальным заданием.

Цель работы обуславливает постановку и решение следующих задач:

1. Изучить особенности файловых систем ext4, Btrfs, ReiserFS, NTFS, FAT32.
2. Освоить создание, настройку и монтирование файловых систем в ОС Linux.
3. Провести экспериментальные измерения на файловых системах ext4, Btrfs, ReiserFS, NTFS и

FAT32. Оценить производительность файловых систем по следующим параметрам:

- Время записи данных в файлы (разного размера).
- Время чтения данных из файлов (разных размеров).
- Время выполнения задачи индивидуального задания для разных входных параметров.

4. Изучить основы работы виртуальной файловой системы в ОС Linux, её роль в абстрагировании работы с файловыми сущностями. Освоить основные механизмы взаимодействия через файловые дескрипторы, а также базовые операции с файловыми системами, реализованные через VFS.

5. Написать программу, в соответствие с индивидуальным заданием №1, реализующую работу

алгоритма с большим файлом (размер зависит от конкретного алгоритма, рекомендуемый размер

— 1–2 ГБ). Язык программирования может быть выбран любой.

6. Разработать пользовательскую файловую систему в соответствии с индивидуальным заданием

№2 с помощью библиотеки FUSE (Filesystem in userspace), позволяющей работать без необходимости модификации ядра ОС (язык реализации — C или Rust).

7. Составить отчет с программами, результатами тестов, измерениями, графиками и выводами.

Вариант 14

Индивидуальные задания

14. Определение частоты появления заданного слова в большом текстовом файле.

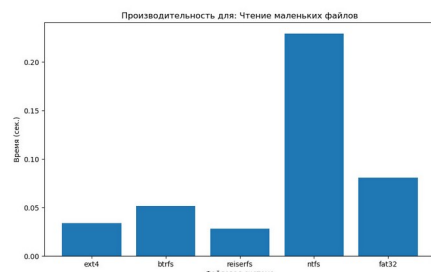
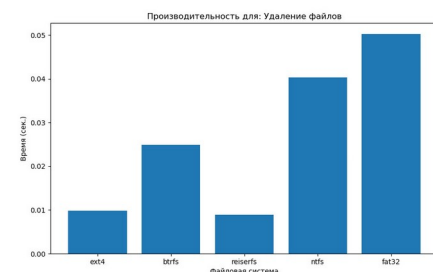
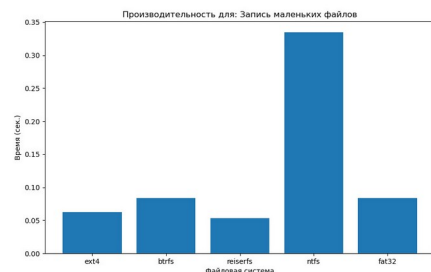
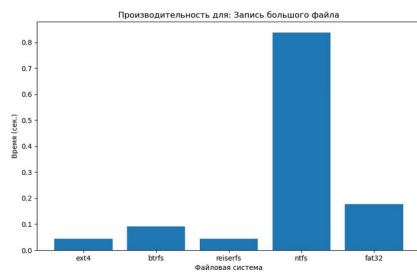
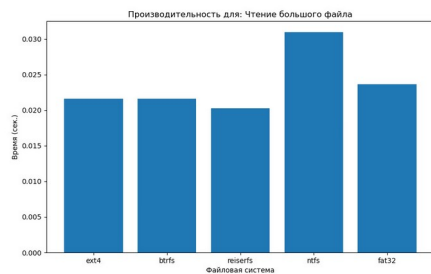
14. Файловая система с автогенерацией метаданных. Реализовать файловую систему, которая автоматически добавляет метаданные к файлам (например, геолокация пользователя, время загрузки и тип устройства, использованного для записи и т.п.).

Выполнение индивидуального задания:

Подготовка к выполнению заданий обуславливает произведение операции монтирования, вот его результаты:

```
~/projects/university/5sem/os/lab3 master* > lsblk -f /dev/loop0
NAME      FSTYPE FSVER LABEL UUID                                 FSAVAIL FSUSE% MOUNTPOINTS
loop0
└─loop0p2 btrfs                                     023a39c7-feac-4a86-a771-e4bd84b9a79a    1,2G    27% /mnt/btrfs
                                     /mnt/btrfs
└─loop0p1 ext4      1.0                                794e3c3f-2907-40dc-8daf-73f87f561aec    1,7G     0% /mnt/ext4
                                     /mnt/ext4
└─loop0p3
└─loop0p4 ntfs                                     18F9A07C625C0D46                        1,9G     1% /mnt/ntfs
└─loop0p5 vfat      FAT32                                ACA9-5513                                1,9G     0% /mnt/fat32
```

Запуск Python скрипты, представленного в лабораторной работе дал следующие результаты



Задание №1

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <sys/time.h>
#include <sys/stat.h>
#include <dirent.h>
#include <unistd.h>
#include <string.h>
#include <ctype.h>

#define LARGE_FILE_SIZE_GB 0.5
#define SMALL_FILES_COUNT 500
#define SMALL_FILE_SIZE_MB 0.5
#define NUM_FS 4
#define WORD_BUFFER_SIZE 100
#define LINE_BUFFER_SIZE 1024

const char* mount_points[NUM_FS] = {
    "/mnt/ext4",
    "/mnt/btrfs",
    "/mnt/ntfs",
    "/mnt/fat32"
};

const char* fs_names[NUM_FS] = {
    "ext4",
    "btrfs",
    "ntfs",
    "fat32"
};

const char* TARGET_WORD = "test";

double get_time() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return tv.tv_sec + tv.tv_usec / 1000000.0;
}

void to_lower_case(char* str) {
    for (int i = 0; str[i]; i++) {
        str[i] = tolower(str[i]);
    }
}

void generate_large_text_file(const char* filename, double size_gb) {
    FILE* file = fopen(filename, "w");
    if (!file) {
        perror("Failed to open large file");
        exit(1);
    }

    long long target_bytes = (long long)(size_gb * 1024LL * 1024LL * 1024LL);
    long long current_bytes = 0;

    const char* sample_words[] = {
        "test", "hello", "world", "file", "system", "linux", "program",
        "data", "computer", "network", "memory", "disk", "processor",
        "algorithm", "database", "security", "application", "development"
    };
    int word_count = sizeof(sample_words) / sizeof(sample_words[0]);

    srand(time(NULL));

    while (current_bytes < target_bytes) {
```

```

char line[LINE_BUFFER_SIZE] = "";
int line_length = 0;

int words_in_line = 10 + rand() % 11;
for (int i = 0; i < words_in_line; i++) {
    const char* word = sample_words[rand() % word_count];
    int word_len = strlen(word);

    if (line_length + word_len + 1 < LINE_BUFFER_SIZE) {
        if (line_length > 0) {
            strcat(line, " ");
            line_length++;
        }
        strcat(line, word);
        line_length += word_len;
    }
}

strcat(line, "\n");
fprintf(file, "%s", line);
current_bytes += strlen(line);
}

fclose(file);
}

void generate_small_text_files(const char* directory, int count, double size_mb) {
    char cmd[256];
    sprintf(cmd, "mkdir -p %s", directory);
    system(cmd);

    long long target_bytes_per_file = (long long)(size_mb * 1024LL * 1024LL);

    const char* sample_words[] = {
        "test", "document", "project", "research", "analysis",
        "result", "method", "technique", "evaluation", "performance",
        "measurement", "benchmark", "comparison", "optimization"
    };
    int word_count = sizeof(sample_words) / sizeof(sample_words[0]);

    srand(time(NULL));

    for (int i = 0; i < count; ++i) {
        char filepath[256];
        sprintf(filepath, "%s/small_%d.txt", directory, i);
        FILE* file = fopen(filepath, "w");
        if (!file) {
            exit(1);
        }

        long long current_bytes = 0;
        while (current_bytes < target_bytes_per_file) {
            char line[LINE_BUFFER_SIZE] = "";
            int line_length = 0;

            int words_in_line = 5 + rand() % 11;
            for (int j = 0; j < words_in_line; j++) {
                const char* word = sample_words[rand() % word_count];
                int word_len = strlen(word);

                if (line_length + word_len + 1 < LINE_BUFFER_SIZE) {
                    if (line_length > 0) {
                        strcat(line, " ");
                        line_length++;
                    }
                    strcat(line, word);
                    line_length += word_len;
                }
            }
        }
    }
}

```

```

        strcat(line, "\n");
        fprintf(file, "%s", line);
        current_bytes += strlen(line);
    }

    fclose(file);
}

int count_word_frequency_in_file(const char* filename, const char* target_word) {
    FILE* file = fopen(filename, "r");
    if (!file) {
        return -1;
    }

    int count = 0;
    char word[WORD_BUFFER_SIZE];
    char lower_target_word[WORD_BUFFER_SIZE];

    strcpy(lower_target_word, target_word);
    to_lower_case(lower_target_word);

    while (fscanf(file, "%99s", word) == 1) {
        to_lower_case(word);

        if (strcmp(word, lower_target_word) == 0) {
            count++;
        }
    }

    fclose(file);
    return count;
}

double find_word_frequency_large_file(const char* filename, const char* target_word, int* frequency) {
    double start = get_time();
    *frequency = count_word_frequency_in_file(filename, target_word);
    return get_time() - start;
}

double find_word_frequency_small_files(const char* directory, const char* target_word, int*
total_frequency) {
    double start = get_time();
    *total_frequency = 0;

    DIR* dir = opendir(directory);
    if (!dir) {
        perror("Failed to open directory");
        return -1;
    }

    struct dirent* entry;
    while ((entry = readdir(dir)) != NULL) {
        if (entry->d_type == DT_REG) {
            char filepath[256];
            sprintf(filepath, "%s/%s", directory, entry->d_name);

            int file_frequency = count_word_frequency_in_file(filepath, target_word);
            if (file_frequency >= 0) {
                *total_frequency += file_frequency;
            }
        }
    }

    closedir(dir);
    return get_time() - start;
}

```

```

void cleanup(const char* large_file, const char* small_dir) {
    remove(large_file);
    char cmd[256];
    sprintf(cmd, "rm -rf %s", small_dir);
    system(cmd);
}

int main() {
    printf("поиск частоты слова '%s' в файловых системах\n\n", TARGET_WORD);

    for (int i = 0; i < NUM_FS; ++i) {
        printf("Файловая система: %s\n", fs_names[i]);

        char large_file[256];
        sprintf(large_file, "%s/large_text.txt", mount_points[i]);
        char small_dir[256];
        sprintf(small_dir, "%s/small_text_files", mount_points[i]);

        printf("генерация большого файла\n");
        generate_large_text_file(large_file, LARGE_FILE_SIZE_GB);

        printf("генерация маленьких файлов\n");
        generate_small_text_files(small_dir, SMALL_FILES_COUNT, SMALL_FILE_SIZE_MB);

        int freq_large = 0;
        double time_large = find_word_frequency_large_file(large_file, TARGET_WORD, &freq_large);

        int freq_small = 0;
        double time_small = find_word_frequency_small_files(small_dir, TARGET_WORD, &freq_small);

        printf("Большой файл: время = %.6f сек, частота = %d\n", time_large, freq_large);
        printf("Маленькие файлы: время = %.6f сек, общая частота = %d\n", time_small, freq_small);
        printf("Общая частота: %d\n\n", freq_large + freq_small);

        cleanup(large_file, small_dir);
    }

    return 0;
}

```

Результат работы программы:

```

поиск частоты слова 'test' в файловых системах

Файловая система: ext4
генерация большого файла
генерация маленьких файлов
Большой файл: время = 7.576531 сек, частота = 3864049
Маленькие файлы: время = 3.321451 сек, общая частота = 1971252
Общая частота: 5835301

Файловая система: btrfs
генерация большого файла
генерация маленьких файлов
Большой файл: время = 7.298489 сек, частота = 3862714
Маленькие файлы: время = 3.169422 сек, общая частота = 1969511
Общая частота: 5832225

Файловая система: ntfs
генерация большого файла
генерация маленьких файлов
Большой файл: время = 7.588670 сек, частота = 3860631
Маленькие файлы: время = 3.290763 сек, общая частота = 1968958
Общая частота: 5829589

Файловая система: fat32
генерация большого файла
генерация маленьких файлов
Большой файл: время = 8.024854 сек, частота = 3863219
Маленькие файлы: время = 3.773410 сек, общая частота = 1971791
Общая частота: 5835010

```

Задание №2

```
#define FUSE_USE_VERSION 31
#define _GNU_SOURCE

#include <fuse3/fuse.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <time.h>
#include <unistd.h>

typedef struct {
    char *content;
    size_t size;
    time_t ctime;
    time_t mtime;
    int write_count;
    pid_t creator_pid;
} file_data;

typedef struct node {
    char *name;
    file_data data;
    struct node *next;
} node;

static node *files = NULL;

static node* find_file(const char *path) {
    if (path[0] != '/' || path[1] == '\0') return NULL;
    const char *name = path + 1;
    for (node *n = files; n; n = n->next)
        if (strcmp(n->name, name) == 0)
            return n;
    return NULL;
}

static node* create_file(const char *path) {
    node *n = calloc(1, sizeof(node));
    if (!n) return NULL;
    n->name = strdup(path + 1);
    if (!n->name) { free(n); return NULL; }
    n->data.content = strdup("");
    n->data.ctime = n->data.mtime = time(NULL);
    n->data.creator_pid = getpid();
    n->data.write_count = 0;
    n->data.size = 0;
    n->next = files;
    files = n;
    return n;
}

static int fs_getattr(const char *path, struct stat *st, struct fuse_file_info *fi) {
    (void)fi;
    memset(st, 0, sizeof(*st));

    if (strcmp(path, "/") == 0) {
        st->st_mode = S_IFDIR | 0755;
        st->st_nlink = 2;
        return 0;
    }

    const char *name = path + 1;
    size_t len = strlen(name);

    if (len > 5 && strcmp(name + len - 5, ".meta") == 0) {
        char realname[256];
        snprintf(realname, sizeof(realname), "%.*s", (int)(len - 5), name);
        if (find_file(realname)) {
            st->st_mode = S_IFREG | 0444;
            st->st_nlink = 1;
            st->st_size = 1024;
            return 0;
        }
        return -ENOENT;
    }

    node *n = find_file(path);
```



```

    if (!n) return -ENOENT;

    st->st_mode = S_IFREG | 0644;
    st->st_nlink = 1;
    st->st_size = n->data.size;
    st->st_ctime = n->data.ctime;
    st->st_mtime = n->data.mtime;
    return 0;
}

static int fs_readdir(const char *path, void *buf, fuse_fill_dir_t filler,
                     off_t offset, struct fuse_file_info *fi, enum fuse_readdir_flags flags) {
    (void)offset; (void)fi; (void)flags;
    if (strcmp(path, "/") != 0) return -ENOENT;

    filler(buf, ".", NULL, 0, 0);
    filler(buf, "..", NULL, 0, 0);

    for (node *n = files; n; n = n->next) {
        filler(buf, n->name, NULL, 0, 0);
        char meta[512];
        snprintf(meta, sizeof(meta), "%s.meta", n->name);
        filler(buf, meta, NULL, 0, 0);
    }
    return 0;
}

static int fs_open(const char *path, struct fuse_file_info *fi) {
    (void)fi;
    if (strstr(path, ".meta")) {
        char realpath[256];
        snprintf(realpath, sizeof(realpath), "%.s", (int)strlen(path)-5, path);
        return find_file(realpath) ? 0 : -ENOENT;
    }
    return find_file(path) ? 0 : -ENOENT;
}

static int fs_read(const char *path, char *buf, size_t size, off_t offset, struct fuse_file_info *fi) {
    (void)fi;

    if (strstr(path, ".meta")) {
        char realpath[256];
        snprintf(realpath, sizeof(realpath), "%.s", (int)strlen(path)-5, path);
        node *n = find_file(realpath);
        if (!n) return -ENOENT;

        char meta[1024];
        snprintf(meta, sizeof(meta),
            "Файл: %s\n"
            "Создан: %s"
            "Изменён: %s"
            "Записей: %d\n"
            "Размер: %zu байт\n"
            "PID создателя: %d\n",
            n->name,
            ctime(&n->data.ctime),
            ctime(&n->data.mtime),
            n->data.write_count,
            n->data.size,
            (int)n->data.creator_pid);

        size_t len = strlen(meta);
        if (offset >= (off_t)len) return 0;
        if (offset + size > len) size = len - offset;
        memcpy(buf, meta + offset, size);
        return size;
    }

    node *n = find_file(path);
    if (!n || offset >= (off_t)n->data.size) return 0;
    if (offset + size > n->data.size) size = n->data.size - offset;
    memcpy(buf, n->data.content + offset, size);
    return size;
}

static int fs_write(const char *path, const char *buf, size_t size,
                   off_t offset, struct fuse_file_info *fi) {
    (void)fi;
    if (strstr(path, ".meta")) return -EACCES;

    node *n = find_file(path);

```

```

    if (!n) n = create_file(path);

    if (offset + size > n->data.size) {
        char *new = realloc(n->data.content, offset + size + 1);
        if (!new) return -ENOMEM;
        n->data.content = new;
        if (offset > (off_t)n->data.size)
            memset(n->data.content + n->data.size, 0, offset - n->data.size);
        n->data.size = offset + size;
    }
    memcpy(n->data.content + offset, buf, size);
    n->data.content[n->data.size] = '\0';
    n->data.mtime = time(NULL);
    n->data.write_count++;
    return size;
}

static int fs_create(const char *path, mode_t mode, struct fuse_file_info *fi) {
    (void)mode; (void)fi;
    if (find_file(path)) return -EEXIST;
    return create_file(path) ? 0 : -ENOMEM;
}

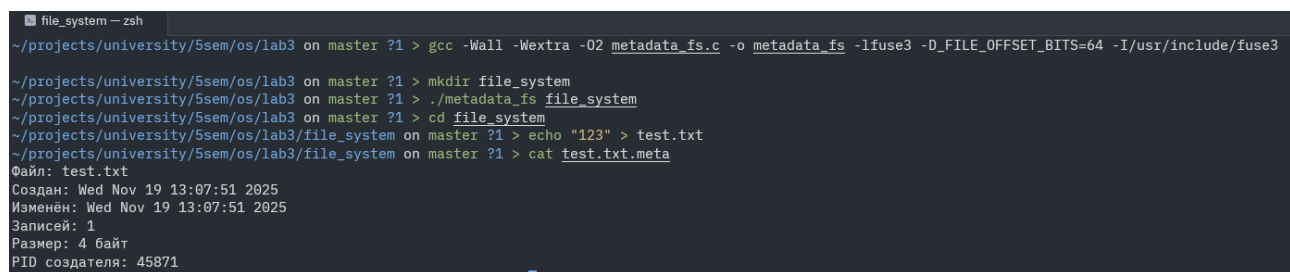
static int fs_unlink(const char *path) {
    if (strstr(path, ".meta")) return -EACCES;
    node **p = &files;
    while (*p) {
        if (strcmp((*p)->name, path + 1) == 0) {
            node *tmp = *p;
            *p = tmp->next;
            free(tmp->name);
            free(tmp->data.content);
            free(tmp);
            return 0;
        }
        p = &(*p)->next;
    }
    return -ENOENT;
}

static const struct fuse_operations ops = {
    .getattr = fs_getattr,
    .readdir = fs_readdir,
    .open = fs_open,
    .read = fs_read,
    .write = fs_write,
    .create = fs_create,
    .unlink = fs_unlink,
};

int main(int argc, char *argv[]) {
    return fuse_main(argc, argv, &ops, NULL);
}

```

Результат работы программы:



```

file_system - zsh
~/projects/university/5sem/os/lab3 on master ?1 > gcc -Wall -Wextra -O2 metadata_fs.c -o metadata_fs -lfuse3 -D_FILE_OFFSET_BITS=64 -I/usr/include/fuse3
~/projects/university/5sem/os/lab3 on master ?1 > mkdir file_system
~/projects/university/5sem/os/lab3 on master ?1 > ./metadata_fs file_system
~/projects/university/5sem/os/lab3 on master ?1 > cd file_system
~/projects/university/5sem/os/lab3/file_system on master ?1 > echo "123" > test.txt
~/projects/university/5sem/os/lab3/file_system on master ?1 > cat test.txt.meta
Файл: test.txt
Создан: Wed Nov 19 13:07:51 2025
Изменён: Wed Nov 19 13:07:51 2025
Записей: 1
Размер: 4 байт
PID создателя: 45871

```

Вывод: в ходе лабораторной работы мы познакомились с механизмами работы с файловой системой, экспериментально проверили работы различных файловых систем и реализовали свою файловую систему, которая хранит все файлы в зжатом виде